LABORATORIO A

PROGRAMACIÓN CON PTHREADS: REDUCCIONES, HISTOGRAMA Y SINCRONIZACIÓN

1. Propósito

Aplicar programación multithreading con Pthreads para resolver problemas computacionales que requieren distribuir el trabajo entre varios threads, compartir datos y sincronizar accesos.

2. Objetivos

- Crear y gestionar threads utilizando Pthreads.
- Distribuir un conjunto de datos entre varios threads.
- Comparar implementaciones secuenciales y multithreading.
- Medir tiempo de ejecución, speedup y eficiencia.
- Identificar condiciones de carrera y aplicar mecanismos de sincronización.
- Explicar cómo la estrategia de partición del trabajo afecta el rendimiento.

3. Consideraciones generales

- Todos los programas deben implementarse en C utilizando Pthreads.
- Para cada problema, implementar primero una versión secuencial de referencia.
- Comprobar que la versión paralela produce resultados correctos.
- Probar, cuando sea razonable, con 1, 2, 4 y 8 threads.
- Registrar el tamaño de los datos, número de threads y características básicas del equipo.
- No basta con presentar el código: explicar la estrategia de paralelización y analizar los resultados.

4. Parte I — Estadísticas de un conjunto de datos

Dado un arreglo de N números enteros, calcular la suma total, el mínimo, el máximo y el promedio. La versión paralela debe dividir el arreglo en segmentos y asignar un segmento a cada thread.

Actividades

- Implementar la solución secuencial.
- Dividir el arreglo entre los threads, contemplando que N no sea divisible exactamente por la cantidad de threads.
- Cada thread debe calcular resultados parciales usando variables locales.
- Combinar los resultados parciales para obtener la suma, el mínimo y el máximo globales.
- Calcular el promedio a partir de la suma total y N.
- Comparar los resultados secuenciales y paralelos para varios tamaños de N.
- Registrar los tiempos de ejecución y calcular speedup y eficiencia.

Preguntas de análisis

- ¿Por qué conviene calcular resultados parciales en cada thread?
- ¿Qué problema surgiría si todos los threads actualizaran una única variable de suma sin sincronización?
- ¿Es necesario utilizar un mutex para combinar los resultados parciales? Justifique su estrategia.
- ¿Cómo se distribuye el trabajo cuando N no es múltiplo del número de threads?
- ¿Por qué el speedup puede ser menor que el número de threads?

5. Parte II — Construcción paralela de un histograma

Dado un arreglo de N valores enteros entre 0 y 99, construir un histograma con diez categorías: 0–9, 10–19, ..., 90–99. Cada posición del histograma indica cuántos valores pertenecen a su intervalo.

Actividades

- Implementar el histograma secuencial.
- Implementar una versión paralela en la que cada thread procese una parte del arreglo.
- Diseñar una estrategia para evitar que varios threads incrementen simultáneamente la misma posición del histograma.
- Como alternativa, implementar un histograma privado por thread y combinar los histogramas al finalizar.
- Verificar que la suma de las diez categorías sea igual a N.
- Comparar ambas estrategias paralelas, si se implementan, en términos de correctitud y tiempo.

Preguntas de análisis

- ¿En qué situación se produce una condición de carrera al actualizar el histograma?
- ¿Qué diferencias hay entre usar un mutex compartido y un histograma privado por thread?
- ¿Qué estrategia esperarías que escale mejor y por qué?
- ¿Cómo influye la distribución de los valores de entrada en la contención?

6. Parte III — Productores y consumidores con buffer acotado

Implementar un buffer circular de capacidad fija compartido por threads productores y consumidores. Los productores generan enteros y los insertan en el buffer; los consumidores extraen elementos.

Actividades

- Definir el buffer, los índices de inserción y extracción, y los contadores necesarios.
- Crear al menos dos productores y dos consumidores.
- Proteger el acceso al buffer y sus índices para evitar modificaciones concurrentes inconsistentes.
- Utilizar mecanismos de sincronización apropiados para que los productores esperen cuando el buffer esté lleno y los consumidores cuando esté vacío.
- Contabilizar los elementos producidos y consumidos y verificar que no se pierdan ni se dupliquen.
- Ejecutar pruebas con distintas cantidades de productores, consumidores y tamaños de buffer.

Preguntas de análisis

- ¿Por qué no basta con proteger únicamente el contador de elementos?
- ¿Qué puede ocurrir si un consumidor intenta extraer un elemento de un buffer vacío?
- ¿Qué puede ocurrir si un productor inserta en un buffer lleno?
- ¿Qué función cumplen el mutex y las variables de condición o semáforos elegidos?
- ¿Cómo comprobarías que todos los elementos producidos fueron consumidos exactamente una vez?

7. Entregables

- Código fuente de las tres partes.
- Instrucciones de compilación y ejecución.
- Tablas con resultados experimentales y tiempos.
- Cálculo de speedup y eficiencia para la Parte I y, cuando sea pertinente, la Parte II.
- Respuestas a las preguntas de análisis.
- Conclusiones sobre correctitud, sincronización y rendimiento.

8. Estructura sugerida del informe

- 1. Introducción y entorno de ejecución.
- 2. Estadísticas del conjunto de datos.
- 3. Construcción del histograma.

- 4. Productores y consumidores.
- 5. Resultados y análisis.
- 6. Problemas encontrados y soluciones.
- 7. Conclusiones.